# LTEM: LLM-Friendly Table-Enhanced Markdown for Table Understanding and Generation

**Eunbin Kim, Hyeonseok Kang, Sangkeun Jung***
Department of Computer Science and Engineering,
Chungnam National University, Republic of Korea
{eunbinkim777,dnfldjaak11,hugmanskj}@gmail.com

## Abstract

Tables in real-world documents, including scientific papers, financial reports, and statistical summaries, frequently use merged cells, multi-level headers, and sectioned layouts. Yet the lightweight formats LLMs handle best (Markdown, CSV) cannot express these structures, while expressive formats (HTML, LaTeX) impose excessive token cost and parsing complexity. To enable systematic analysis of this trade-off, we introduce a **taxonomy** decomposing table structure into six atomic features (F1–F6) and corresponding complexity levels (L1–L6), and construct **STB** (Synthetic Table Benchmark), a controlled benchmark with 1,089 tables systematically distributed across complexity levels. Using this framework, we propose **LTEM** (LLM-friendly Table-Enhanced Markdown), a formally specified, minimal extension of Markdown that explicitly encodes spans and hierarchical headers. Experiments across five state-of-the-art LLMs show that LTEM achieves accuracy comparable to HTML, offers a favorable accuracy–token trade-off relative to HTML and LaTeX, and produces substantially fewer invalid responses than LaTeX; for table generation, LTEM is comparable to MultiMarkdown though HTML is best. We release our taxonomy, benchmark, and format specification to support future research on LLM-table interaction.

## 1 Introduction

LLMs excel at natural language tasks, yet many practical applications, including document editing, report writing, and data summarization, require tables. Because tables are two-dimensional while LLM inputs are linear, models must reconstruct header roles, spans, and hierarchies from serialized text, making performance sensitive to the *representation format*.

Recent work confirms that format choice matters: identical tables yield different performance depending on serialization, delimiters, and ordering (Sui et al., 2024; Li et al., 2025; Wu et al., 2025). However, existing studies provide limited tools for comparing *what structural information* each format preserves.

Real-world tables often include merged cells, multi-level headers, and sectioned layouts. Lightweight formats (Markdown, CSV) are easy for LLMs but cannot faithfully express these structures, while expressive formats (HTML, LaTeX) are verbose and increase token cost and parsing burden. For example, GitHub-Flavored Markdown (GFM) defines pipe-delimited tables (GitHub, 2019) but does not support `colspan`/`rowspan` (MDN Web Docs, 2026). MultiMarkdown adds column-spanning syntax through repeated pipes (Penney, 2023). In contrast, HTML and LaTeX can represent complex tables but require substantial markup interpretation.

To systematize this trade-off, we define six **Structural Features (F1–F6)** and **Complexity Levels (L1–L6)** as cumulative feature requirements. This taxonomy makes table complexity explicit and enables direct comparison of formats by expressiveness.

We then propose **LTEM**[1] (LLM-friendly Table-Enhanced Markdown), a minimal extension of Markdown that explicitly encodes spans and multi-level headers while remaining linear to read and parsable in a single pass. LTEM targets L3–L5 structures with less verbosity than HTML/LaTeX, aiming to improve token efficiency and response stability without sacrificing expressiveness.

We evaluate LTEM on **Table Understanding** (Table QA) and **Table Generation** across formats (Markdown, CSV, HTML, LaTeX, MultiMarkdown, LTEM) and five state-of-the-art LLMs, and analyze **Token Efficiency** using token-normalized metrics.

*Corresponding author.

[1] https://github.com/hugman/ltem



*Proceedings of the 2026 Conference on Empirical Methods in Natural Language Processing*, pages 43304–43324
October 24-29, 2026 ©2026 Association for Computational Linguistics

Our contributions are as follows:

- **Structural complexity taxonomy**: We formalize table difficulty and format expressiveness via F1–F6 and L1–L6, providing a systematic framework for comparing serialization capabilities.
- **STB benchmark**: We construct a controlled benchmark with 1,089 tables systematically distributed across complexity levels, enabling controlled analysis of structural effects on LLM performance.
- **LTEM**: We propose a minimal, formally specified Markdown extension that instantiates the taxonomy-derived requirements, explicitly encoding cell spans and hierarchical headers while preserving LLM-friendly linear readability.
- **Empirical trade-off analysis**: We evaluate understanding, generation, and token efficiency across formats and diverse LLM backbones.

## 2 Related Work

### 2.1 Table QA and Table Reasoning

Early Table QA focused on semantic parsing, mapping questions to executable programs. WTQ (Pasupat and Liang, 2015) and WikiSQL (Zhong et al., 2017) established QA over Wikipedia tables, and later work introduced pretrained table encoders such as TAPAS and TaBERT (Herzig et al., 2020; Yin et al., 2020). However, these datasets largely emphasize flat tables, underrepresenting document-style structures such as merged cells and hierarchical headers. Benchmarks including HiTab and TAT-QA better reflect hierarchical and numerically grounded reasoning (Cheng et al., 2022; Zhu et al., 2021). In contrast to work centered on tasks or model variants, we treat **serialization format** as a primary variable and study its impact on stability and token efficiency across structural complexity.

### 2.2 LLMs on Tables and Format Choice

Recent studies show that LLM performance on tabular tasks is sensitive to serialization, delimiters, and ordering (Sui et al., 2024; Fang et al., 2024; Lu et al., 2025). LongTableBench emphasizes format and context length in long-context table reasoning (Li et al., 2025), and RealHiTBench compares representations for complex hierarchical tables across LaTeX, HTML, and images (Wu et al., 2025). While these works establish that format matters, they offer limited tools to compare *what structural information* each representation preserves. Our taxonomy addresses this gap by decomposing table structure into atomic operations.

### 2.3 Markup Expressiveness and Text-to-Table Evaluation

Markdown tables are common: GFM defines pipe-delimited tables (GitHub, 2019) but lacks structural attributes such as `colspan`/`rowspan` (MDN Web Docs, 2026), while MultiMarkdown adds column-spanning syntax through repeated pipes (Penney, 2023). HTML and LaTeX provide explicit span constructs for complex tables (WHATWG, 2026; LaTeX2e Unofficial Reference Manual Project, 2026; van Oostrum et al., 2024). Their more verbose syntax motivates our evaluation of format-dependent accuracy and structural consistency. For text-to-table generation, evaluation is difficult because multiple surface forms can represent the same structure. Prior work proposes multi-granularity table metrics (Pal et al., 2023), statement-based checks (Ramu et al., 2024), and discusses structure-aware metrics under constrained decoding (Oestreich and Müller, 2025). We analyze format-dependent structural consistency via normalized representations, following exact-match perspectives common in structured output evaluation (Yu et al., 2018).

## 3 Taxonomy of Table Structural Complexity

An objective comparison of table representation format capabilities requires clear criteria. This section presents a **taxonomy** that serves as an evaluation methodology for systematically characterizing table structural complexity and the structural information a serialization must preserve for LLMs. This classification scheme comprises two axes: **Structural Features (F1–F6)**, the atomic operations for expressing table structure, and **Complexity Levels (L1–L6)**, the cumulative feature requirements for expressing specific tables.

### 3.1 Structural Features

We define six atomic operations for expressing table structure. Each Feature is independent of the others, enabling binary assessment of whether a specific structural capability is present or absent.

| Feature | CSV | MD | MMD | HTML | LaTeX | LTEM | LTEM Syntax |
|---|---|---|---|---|---|---|---|
| F1 Grid Formation | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | `\| \|` |
| F2 Header Designation | △ | ✓ | ✓ | ✓ | ✓ | ✓ | `\|−\|` |
| F3 Cell Merging | ✗ | ✗ | △ | ✓ | ✓ | ✓ | «, ^^, `{c:N,r:M}` |
| F4 Header Nesting | ✗ | ✗ | △ | ✓ | ✓ | ✓ | multi-header rows |
| F5 Section Stacking | ✗ | ✗ | ✗ | △ | ✓ | ✓ | section-label rows |
| F6 Table Composition | ✗ | ✗ | ✗ | △ | ✓ | ✓ | `\| # id \|, @ref` |

Table 1: Structural feature support by format. ✓ = native support; △ = partial or non-standard; ✗ = not supported. Dashed vertical rule separates existing formats from LTEM.

**F1 (Grid Formation).** F1 (Grid Formation) represents the most basic requirement: that cells "exist" at row-column intersections.

**F2 (Header Designation).** F2 (Header Designation) indicates that cells have assigned "roles." In HTML, header cells use `<th>` while data cells use `<td>`; in Markdown, rows above the separator line (`|−|`) are treated as headers.

**F3 (Cell Merging).** F3 (Cell Merging) is the critical branching point in this taxonomy. A substantial portion of tables in scientific papers require cell merging for group headers or category representation.

**F4 (Header Nesting).** F4 (Header Nesting) applies when headers themselves have parent–child hierarchies, exemplified by grouping quarterly revenue as "Q1", "Q2", "Q3" under a "Revenue" superheader.

**F5 (Section Stacking).** F5 (Section Stacking) and F6 (Table Composition) address compound table structures. F5 applies when multiple sections sharing identical column structure (schema) coexist within a single table, such as displaying "Model A Results" and "Model B Results" in one table in regression analysis.

**F6 (Table Composition).** F6 applies when tables with different schemas constitute a single logical unit, such as placing "(a) Correlation Matrix" and "(b) Sample Sizes" in one figure.

### 3.2 Complexity Levels

Complexity Level is defined as the **cumulative features required** to express a specific table. Each Level includes all Features from previous Levels and adds new required Features. Complexity Level definitions are summarized in Table 5 in the Appendix.

The key implication of this cumulative structure is: a format that cannot support a specific Level cannot structurally express any higher Levels. For example, a format not supporting F3 (Cell Merging) cannot structurally express not only L3 but also L4, L5, and L6 tables.

### 3.3 Expressiveness of Existing Formats

We apply the above taxonomy to analyze the expressiveness of major table representation formats. The analysis covers five formats spanning minimal data exchange formats (CSV), lightweight markup with high frequency in LLM training data (Markdown, MultiMarkdown), and document standards with complete structural expressiveness (HTML, LaTeX). Two important patterns emerge from this analysis (Table 1).

**First**, F3 (Cell Merging) is the decisive branching point. Markdown does not support F3 at all, fundamentally precluding expression of L3+ tables. MultiMarkdown supports `colspan` in some renderers but lacks standardization, with even more limited `rowspan` support. This is not merely a feature absence but a structural limitation: the syntax for encoding this information simply does not exist.

**Second**, only LaTeX has complete expressiveness. HTML can approximate F5 and F6 using multiple `<tbody>` elements or separate `<table>` elements, but cannot structurally specify logical relationships between sections. LaTeX is the only format that can express structures at all levels through a nesting-based approach using nested `tabular` environments and `subtable`.

This analysis presents an important implication overlooked in prior research: the format LLMs process most efficiently (Markdown) cannot express L3+ tables commonly found in scientific papers, while formats capable of such expression (LaTeX, HTML) are inefficient for LLM processing.

LaTeX's `\multicolumn{2}{c}{...}` and HTML's `<td colspan="2">` encode cell merging information, but LLMs must undergo additional steps to interpret this syntax and infer actual cell

positions and ranges. The same burden exists when providing tables as images, requiring inference of cell boundaries and merging relationships from visual structure.

This taxonomy-based analysis yields concrete serialization requirements for LLM table processing: a format targeting L3–L5 structures should explicitly encode span information and header hierarchies while maintaining linear readability. This motivates the design of a new table markup language optimized for LLM comprehension.

## 4 LTEM: Language Design

To demonstrate how the taxonomy-derived requirements (Section 3) translate into concrete format design, we present LTEM (**L**LM-friendly **T**able-**E**nhanced **M**arkdown) as an instantiation that explicitly encodes F3–F5 features while preserving Markdown's linear readability. This section describes design principles, syntax specification, and LTEM's position within the landscape presented in Section 3.

### 4.1 Design Principles

LTEM's design is grounded in *five* principles derived from the taxonomy-based analysis in Section 3, each addressing specific structural requirements identified by our framework.

**Markdown Compatibility.** LLM training corpora include substantial proportions of sources using Markdown format, such as GitHub and StackExchange (e.g., GitHub 4.5%, StackExchange 2% in LLaMA; (Touvron et al., 2023)). This composition leads LLMs to exhibit high familiarity with Markdown syntax; in fact, major commercial LLMs such as ChatGPT and Claude use Markdown as their default output format for table generation. LTEM is designed as a superset of Markdown to leverage this existing knowledge. Even when new syntax elements render as plain text in existing Markdown parsers, basic table structure is preserved. This design enables LLMs to understand LTEM as an extension of familiar Markdown rather than learning an entirely new format.

**Linear Readability.** Table structure should be **semantically comprehensible** when reading source text sequentially from top to bottom. This principle aligns with LLMs' autoregressive processing and simultaneously facilitates human review and debugging.

```
| Revenue | <<  |     | Revenue |
| ------- | --- | --- | ------- |
| Q1      | Q2  |     | Q1 Q2   |
```

(a) Motivation: span-aware vs. span-unaware.

```
| Revenue | <<      |   | Revenue {c:2} |
| ------- | ------- | = | ------- | ------- |
| Q1      | Q2      |   | Q1      | Q2      |
```

(b) Equivalent encodings: « vs. `{c:N}`.

Figure 1: Colspan in LTEM. (a) illustrates why explicit span markers matter; (b) shows marker- and attribute-based forms are structurally equivalent.

**Explicit Structure.** Structural information should be expressed through explicit markers rather than implicit rules. For example, a cell with colspan=2 is clearly indicated with a `{c:2}` attribute or « marker, eliminating the burden of contextual inference by LLMs. This blocks a major source of structural hallucination.

**Minimal Grammar Extension.** We minimize the number of new syntax elements. LTEM adds only seven syntax elements to standard Markdown: four for span expression («, `^^`, `{c:N}`, `{r:N}`), one for in-cell line breaks (`\\`), one for table identification (`| # id |`), and one for references (`@`). Appendix G.2 reports a design-choice ablation comparing alternative forms for five LTEM elements.

**Unidirectional Parsing.** While Linear Readability concerns semantic understanding, this principle concerns **syntax parsing implementation**. LTEM is parsable in a single pass without forward references, guaranteeing efficient parser implementation with linear time complexity.

### 4.2 Core Syntax

LTEM's core syntax addresses cell spanning and multi-level headers.

**Colspan.** Column spanning is expressed via marker syntax («) or attribute syntax (`{c:N}`). The two forms are structurally equivalent, as illustrated in Figure 1.

Marker syntax maintains visual alignment, while attribute syntax is efficient for large span values, allowing concise expression like `{c:5}` without repeated marker entry.

**Rowspan.** Row spanning is expressed via the `^^` marker or `{r:N}` attribute. Compound spanning combines as `{c:N,r:M}`.

```
| Report | <<  | <<  |
| Region | Q1  | Q2  |
| ------ | --- | --- |
| Asia   | 100 | 120 |
```

Figure 2: Multi-level headers in LTEM: multiple header rows above the separator line naturally encode hierarchical header nesting (F4).

```
| Feature | Details                          | Status      |
| ------- | -------------------------------- | ----------- |
| Speed   | - 2x faster \\ - 50% less memory | [x] Done    |
| Formula | $E = mc^2$                       | [ ] Pending |
```

Figure 3: Rich cell content in LTEM: inline line breaks (\\) enable structured content (lists, checkboxes) and math within a single cell.

**Multi-header Rows.** All rows above the separator line (|−|) are treated as headers. This design naturally supports F4 (Header Nesting) without additional syntax (Figure 2).

**Multiline Cell.** The \\ escape sequence represents line breaks within cells.

### 4.3 Rich Cell Content

LTEM supports structured content within cells based on \\. Bullet lists, numbered lists, checkboxes, and formulas can be combined within a single cell, as illustrated in Figure 3.

### 4.4 Relational Tables

LaTeX and HTML permit nested structures inserting tables within cells, but this requires recursive parsing, disrupts linear reading, and expresses inter-table relationships only implicitly. To address this, LTEM adopts an alternative approach: **table decomposition with explicit references**, as illustrated in Figure 4. This is analogous to normalization principles in relational databases.

A table is identified by a unique label placed in the first row as `| # identifier |`. Other tables are referenced with the `@` prefix.

### 4.5 Formal Specification

LTEM syntax is defined as a Context-Free Grammar (CFG). The complete EBNF (Extended Backus-Naur Form) specification is provided in Appendix C, together with syntax validation rules (V1–V12) and AST and TableStructure JSON schemas.

```
|# orders |
| OrderID | Customer | Items       | ShipTo |
| ------- | -------- | ----------- | ------ |
| 1001    | Alice    | @items_1001 | Seoul  |
| 1002    | Bob      | @items_1002 | Busan  |

|# items_1001 |
| Product | Qty | UnitPrice |
| ------- | --- | --------- |
| Apple   | 3   | 1.00      |
| Banana  | 5   | 0.50      |

|# items_1002 |
| Product | Qty | UnitPrice |
| ------- | --- | --------- |
| Laptop  | 1   | 999.00    |
```

Figure 4: Relational tables in LTEM: tables are decomposed and linked via explicit table IDs (`| # id |`) and references (`@ref`) instead of nesting.

## 5 Experimental Setup

### 5.1 Experimental Design

The experiments are designed to answer the following research questions:

- **RQ1**: Can LTEM sufficiently express diverse tables?
  - **RQ1-1**: Does LTEM improve LLM table comprehension?
  - **RQ1-2**: Does LTEM achieve high fidelity in table generation?
- **RQ2**: Is LTEM more token-efficient than existing formats?

To address these questions, we utilize two complementary datasets. First, the Synthetic Table Benchmark (STB) provides synthetic tables with systematically controlled structural complexity, enabling controlled analysis of format effects at each Complexity Level. Second, existing public benchmarks such as HiTab (Cheng et al., 2022) ensure comparability with prior research.

### 5.2 Synthetic Table Benchmark (STB)

Existing table benchmarks were collected for specific tasks like Table QA or Table-to-Text, without systematic control of table structural complexity. Most tables in WikiTableQuestions have simple header–data structures (L2), with only a minority containing cell merging or multi-level headers. This distributional bias implies that conclusions like "format A outperforms format B" may only hold within specific complexity ranges.

To overcome this limitation, we constructed the Synthetic Table Benchmark (STB), a synthetic

dataset systematically distributed according to the Complexity Levels (L1–L6) defined in Section 3. STB comprises 1,089 tables and 4,356 question–answer pairs, with each table containing an average of 4 QA pairs.

STB's complexity distribution was deliberately designed for research purposes (Table 6). Notably, approximately 30% is allocated to L3, the first level requiring Cell Merging and the boundary where Markdown and CSV become structurally incapable of representation. We anticipated LTEM's differential value would be most evident at this point. To ensure content realism, we defined five domains: *Scientific, Financial, Product, Survey*, and *Schedule* (Table 7), with complexity levels and domains sampled independently to prevent specific domains from being biased toward specific complexity levels.

Table generation follows a 3-stage pipeline: (i) rule-based determination of table structure (row/column counts and merging patterns), (ii) Claude Sonnet 4.5 and Opus 4.5 (Anthropic, 2025b,a) populate domain-appropriate headers and cell values, then (iii) GPT-5-mini generates question–answer pairs (OpenAI, 2025). Question types comprise LOOKUP, COUNTING, COMPARISON, and AGGREGATION, referencing operation types repeatedly used in prior Table QA research (Pasupat and Liang, 2015; Cheng et al., 2022). Additional implementation details and prompt templates are provided in Appendices E and D.

Since automatically generated QA may contain errors, we adopted an indirect verification approach using **model agreement** rather than explicitly computing or verifying answer labels with rules. Specifically, we input identical tables and questions to two models from different families (Qwen3-VL-8B-Instruct and Phi-4-mini-instruct (Qwen Team, 2025b; Microsoft, 2025)), retaining QA samples only when both models' outputs match the GPT-5-mini-generated answer under the same normalization criteria. This agreement serves as a reliability filter, although errors shared across models may remain.

### 5.3 Table QA Benchmarks

To verify whether trends observed in the synthetic dataset (STB) generalize to real-world tables, we used HiTab (Cheng et al., 2022), a public benchmark for hierarchical table question answering. HiTab contains tables extracted from Wikipedia and statistical reports, featuring multi-level headers, cell merging, and hierarchical structure properties underrepresented in existing Table QA benchmarks.

We used HiTab's **test split** to evaluate two distinct tasks.

First, for the Table Understanding Task, we use one original HiTab question–answer pair per table to evaluate how effectively different table representation formats convey complex table structures to LLMs. Each table is converted to six table representation formats (Markdown, CSV, HTML, LaTeX, MultiMarkdown, LTEM) while preserving original structure and content, with evaluation conducted using question–answer pairs for each format.

Second, as an additional task, we evaluate table generation performance from natural language inputs based on the same HiTab tables. Natural language generation requests are generated using GPT-5 based on each table's structure and values, with format-specific generation results analyzed through comparison with pre-generated reference tables for six table representation formats.

### 5.4 Models and Evaluation Metrics

**Evaluation Models.** To assess the consistency of format effects across model families, we conduct Table Understanding experiments with five LLMs and Table Generation experiments with three. Closed-weight models include GPT-5 and Gemini 2.5 Pro (OpenAI, 2025; Google DeepMind, 2025). Open-weight models include the multimodal general-purpose Llama-4-Maverick-17B-128E-Instruct (Meta AI, 2025), the large-parameter Qwen3-235B-A22B-Instruct-2507 (Qwen Team, 2025a), and the reasoning-specialized DeepSeek-R1 (DeepSeek-AI, 2025).

Task-specific prompt templates are held constant across models and formats, with format-dependent variation restricted to the table serialization or target output format and, where applicable, the corresponding syntax guidance. Temperature is set to each model's default value.

**Table Understanding Evaluation.** Table QA performance is primarily evaluated with normalized Exact Match (nEM), and we additionally report token-level F1 (tokenF1) to capture partial-match behavior (Pasupat and Liang, 2015; Cheng et al., 2022; Rajpurkar et al., 2016). Following numeric-answer evaluation practice in table QA (Zhu et al., 2021), we compute numeric Exact

Match (numEM) by extracting numbers and checking equality within a tolerance to reduce notation-induced discrepancies. To assess format stability, we decompose errors into Refusal (explicit answer avoidance) and Invalid (unnormalizable output or parsing failure); both are counted as incorrect, and their sum is reported as Total Error.

**Table Generation Evaluation.** For Table Generation, we prompt models to reconstruct HiTab tables *from natural-language descriptions only* in a specified output format (Markdown/CSV/HTML/LaTeX/MultiMarkdown/LTEM). Unlike Table Understanding, *exact format-level reproduction* is highly challenging, so we adopt **ANLS** (Average Normalized Levenshtein Similarity) as the primary metric to capture graded similarity between generated outputs and reference tables (Biten et al., 2019). We additionally report **Good+** (ANLS $\geq$ 0.7) and **Excellent** (ANLS $\geq$ 0.9) rates as practical success criteria; the full Excellent-rate breakdown is provided in Appendix F.

**Token Efficiency Evaluation.** Since input length varies by serialization format, we also report token-normalized metrics normalizing performance by input token count. For model $m$, format $f$, and benchmark/setting $s$, let input prompt token count be $Tok_{\text{prompt}}(m, f, s)$. When format-specific fixed-length grammar guides are included (e.g., LTEM), we separate this as a constant term:

$$Tok^{\text{adj}}_{\text{prompt}}(m, f, s) = Tok_{\text{prompt}}(m, f, s) - g_f$$

Token-normalized performance for Table Understanding is then:

$$\text{TNP}_{\text{adj}}(m, f, s) = 1000 \times \frac{\text{nEM}(m, f, s)}{Tok^{\text{adj}}_{\text{prompt}}(m, f, s)}$$

This token/cost budget perspective for efficiency comparison is related to prior efficiency analyses (Han et al., 2025; Zhang et al., 2025, 2024; Jung and Kim, 2024).

**Statistical Validation.** For Table Understanding (QA), we performed three repeated runs per setting (model $\times$ format) on identical instances. We applied McNemar's test (McNemar, 1947) on paired predictions (Fisher-combined across runs) and report mean performance over runs, interpreting only consistent differences as significant.

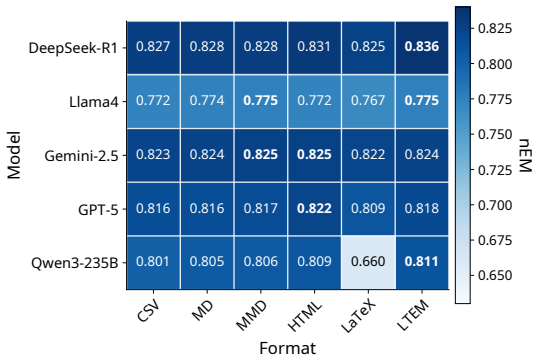



Figure 5: Model $\times$ serialization-format performance (nEM) on STB. Format sensitivity varies by model; notably, Qwen3-235B exhibits a large degradation under LaTeX serialization.

## 6 Results

### 6.1 Table Understanding

In this section, we analyze how table serialization formats affect Table QA accuracy and response stability. We first examine format effects on STB under controlled structural conditions, with each table–question instance serialized in all six formats. We then assess whether the observed trends extend to HiTab, a real-world hierarchical-table benchmark with a substantially different distribution of table sizes and structural properties.

On STB, LTEM achieves nEM comparable to HTML (0.813 vs. 0.812), while LaTeX is lower at 0.777 (Table 2). The gap is clearer in the error breakdown: LTEM keeps Refusal at 0.15% and Invalid at 0.14%, yielding the lowest Total Error, whereas LaTeX shows a sharp increase in Invalid to 2.67% (Table 3). Figure 5 further shows that some models are particularly sensitive to LaTeX serialization, whereas LTEM remains stable under the same evaluation setting. Relative to HTML and LaTeX, LTEM provides a favorable accuracy–token trade-off (Table 2). McNemar tests show that LTEM significantly outperforms LaTeX for four of five models (Fisher-combined $p < 0.01$); Gemini-2.5-Pro shows no significant difference (Appendix H). Failure-mode and question-type breakdowns are in Appendix G.4.

When broken down by Complexity Level, performance declines from L2 to L3 and further at L4 (Figure 6). These transitions reflect the point where models must reliably track span ranges and header roles as cell merging and multi-level headers appear. LTEM is designed to reduce ambiguity at this boundary by explicitly encoding spans and

| Format | nEM↑ | TokenF1↑ | NumEM↑ | PromptTok↓ | TableTok↓ | $\text{TNP}_{\text{adj}}$ ↑ |
|---|---|---|---|---|---|---|
| **LTEM** | **0.813**$_{\pm 0.002}$ | 0.844$_{\pm 0.002}$ | 0.943$_{\pm 0.000}$ | 449 | 228 | 1.809 |
| HTML | **0.812**$_{\pm 0.001}$ | 0.844$_{\pm 0.000}$ | 0.945$_{\pm 0.000}$ | 612 | 392 | 1.326 |
| MMD | 0.810$_{\pm 0.000}$ | 0.843$_{\pm 0.000}$ | 0.946$_{\pm 0.001}$ | 436 | 215 | 1.858 |
| MD | 0.810$_{\pm 0.000}$ | 0.842$_{\pm 0.000}$ | 0.945$_{\pm 0.001}$ | 437 | 217 | 1.852 |
| CSV | 0.808$_{\pm 0.001}$ | 0.840$_{\pm 0.001}$ | 0.940$_{\pm 0.000}$ | 370 | 149 | **2.183** |
| LaTeX | 0.777$_{\pm 0.001}$ | 0.812$_{\pm 0.001}$ | 0.933$_{\pm 0.001}$ | 447 | 227 | 1.737 |

Table 2: Overall STB performance by format (average over 5 models, 3 runs). PromptTok excludes LTEM guide tokens; $\text{TNP}_{\text{adj}} = \text{nEM/PromptTok} \times 1000$. LTEM and HTML achieve comparable nEM. Shorter formats obtain higher token-normalized scores, while LTEM provides a favorable accuracy–token trade-off relative to HTML and LaTeX.

| Format | Refusal (%) | Invalid (%) | Total Error (%) |
|---|---|---|---|
| **LTEM** | **0.15** | 0.14 | 0.29 |
| CSV | 0.22 | 0.09 | 0.31 |
| HTML | 0.30 | 0.11 | 0.41 |
| MMD | 0.34 | 0.15 | 0.49 |
| MD | 0.46 | 0.15 | 0.61 |
| **LaTeX** | 0.38 | **2.67** | **3.04** |

Table 3: Error rate analysis on STB. LTEM shows the lowest refusal rate. LaTeX exhibits a markedly higher invalid response rate than the other formats, primarily due to formatting and parsing failures.

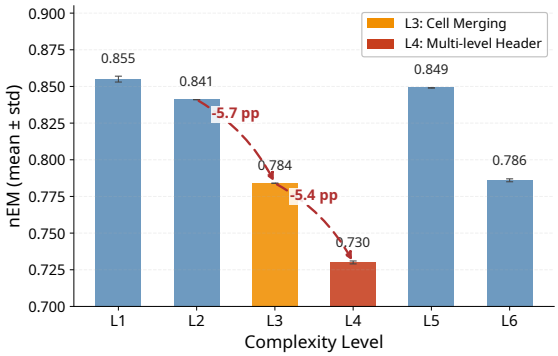



Figure 6: Average performance (nEM) by structural complexity level (L1–L6) on STB. Arrow labels report absolute nEM changes in percentage points (pp). The highlighted L2–L3 and L3–L4 transitions correspond to the introduction of cell merging and multi-level headers.

header nesting. Structure-conditioned results are reported in Appendix G.3. Additional level- and feature-wise breakdowns are in Appendix G.1.

On HiTab, absolute accuracy decreases across formats and rankings become more dependent on dataset characteristics (Figure 7). Average nEM is 0.295 for HTML, 0.292 for LaTeX, and 0.289 for LTEM, with a small 0.6 percentage-point gap (full results in Appendix F). Overall, on STB, LTEM achieves accuracy comparable to HTML and has a substantially lower Invalid rate than LaTeX (Tables 2 and 3).

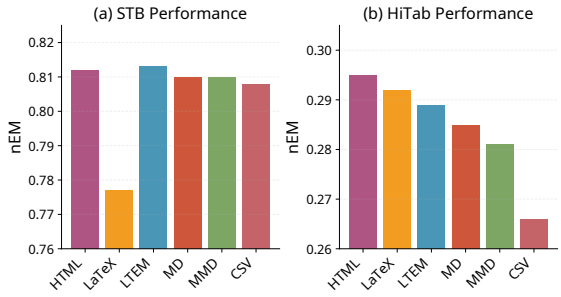



Figure 7: Format-wise performance comparison (nEM) between STB and HiTab. The relative ordering of formats changes across datasets.

### 6.2 Table Generation

For the **Table Generation** task, we use HiTab tables as targets and evaluate whether a model can reconstruct the same tables *from natural-language descriptions only* in a specified output format. Generation quality is evaluated using **ANLS** (normalized string similarity) and the **Good+** rate (ANLS $\geq 0.7$), while **PromptTok** denotes the token count of the user prompt.

As shown in Table 4, HTML achieves the best generation quality (ANLS 0.913) and the highest Good+ rate (96.2%). LaTeX, MultiMarkdown, and LTEM form a similar middle-performing group, while Markdown and CSV perform worse as they cannot directly express complex structures (e.g., merged cells and hierarchical headers).

LTEM is designed to encode complex structures (F3–F4) rather than maximize generation scores; limited pretraining exposure constrains short-term performance, yet it remains close to MultiMarkdown (ANLS 0.720 vs. 0.734), suggesting it is a viable alternative in structure-heavy pipelines.

## 7 Conclusion

This paper introduced a **structural complexity taxonomy** (F1–F6, L1–L6) and **STB**, a controlled

| Format | ANLS↑ | Good+ (%) | PromptTok↓ |
|---|---|---|---|
| HTML | **0.913** | **96.2** | 1149 |
| LaTeX | 0.749 | 72.2 | 1168 |
| MultiMarkdown | 0.734 | 69.0 | 1170 |
| **LTEM** | 0.720 | 67.9 | 1429 |
| CSV | 0.618 | 50.0 | 1170 |
| Markdown | 0.549 | 41.2 | 1168 |

Table 4: Table Generation performance (HiTab average; ANLS: normalized string similarity). PromptTok includes format-specific generation instructions, with an additional syntax guide for LTEM.

benchmark for analyzing how serialization formats affect LLM performance across complexity levels. We then proposed **LTEM**, a minimal Markdown extension that explicitly encodes cell spans and hierarchical headers, instantiating taxonomy-derived requirements as concrete syntax choices. Experiments on STB and HiTab show that LTEM achieves competitive table understanding accuracy while improving response stability at structural transitions where lightweight formats fail and verbose formats introduce noise. Beyond LTEM itself, we hope the taxonomy and benchmark provide reusable infrastructure for future research on robust and efficient LLM–table interaction.

## Limitations

- **Dependence on data distribution.** Because the ranking of formats differs between STB (synthetic) and HiTab (real-world), it is difficult to generalize the superiority of a particular serialization format from a single benchmark. In practice, formats should be re-evaluated on a validation set whose distribution matches the target domain.

- **Training Distribution Bias.** LLM pretraining corpora contain substantially more HTML and Markdown tables than LTEM-formatted content, as LTEM is introduced in this work. This exposure imbalance may favor established formats, particularly in generation tasks that depend on learned serialization patterns. HTML's strong generation performance (ANLS: 0.913, 76% Excellent) may partly reflect this imbalance. In a separate table-QA setting, Appendix G.5 reports a small LoRA pilot showing that 4B-scale models can adapt to LTEM with limited supervision.

- **Model–format interactions.** Format effects vary substantially across models, and some model–format pairs exhibit atypical behaviors such as elevated invalid-response rates. Therefore, format selection cannot be made independently of the model, and prior validation on the deployment backbone is necessary.

- **Sensitivity to experimental settings (prompting/decoding).** Our experiments are based on a specific prompt template and decoding configuration. In particular, output length constraints, instruction wording, and post-processing rules can affect invalid/refusal rates for verbose formats such as LaTeX, and relative rankings may change under different settings.

- **Limited coverage of formats and domains.** While we compare six representative formats, the space of possible serialization designs is broader. Moreover, HiTab focuses primarily on Wikipedia tables, so our conclusions may not directly transfer to other industrial domains (e.g., OCR-extracted tables or noisy financial/log tables) without additional verification.

## Acknowledgments

This work was supported by the National Research Foundation of Korea (NRF) grant funded by the Korean government (MSIT) (No. RS-2025-00556217314820926401 01); the Institute of Information & Communications Technology Planning & Evaluation (IITP) grant funded by the Korean government (MSIT) (RS-2019-II190004, *Development of Semi-supervised Learning Language Intelligence Technology and Korean Tutoring Service for Foreigners*); a grant (26212MFDS008) from the Ministry of Food and Drug Safety in 2026; the Institute of Information & Communications Technology Planning & Evaluation (IITP) grant funded by the Korean government (MSIT) (RS-2026-25569031, *Development of an AIBOM-Based Platform for Strengthening the Reliability and Security of the AI Supply Chain*); and the BK21 FOUR Program by Chungnam National University Research Grant, 2026.

## References

Anthropic. 2025a. System card: Claude opus 4.5. https://www-cdn.anthropic.com/bf10f64990cfda0ba858290be7b8cc6317685f47/Claude%20Opus%204.5%20System%20Card.pdf.

Anthropic. 2025b. System card: Claude sonnet 4.5. https://www-cdn.anthropic.com/963373e433e489a87a10c823c52a0a013e9172dd/Claude%20Sonnet%204.5%20System%20Card.pdf.

Ali Furkan Biten, Ruben Tito, Andres Mafla, Lluis Gomez, Marçal Rusiñol, Ernest Valveny, C. V. Jawahar, and Dimosthenis Karatzas. 2019. Scene text visual question answering. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 4291–4301.

Zhoujun Cheng, Haoyu Dong, Zhiruo Wang, Ran Jia, Jiaqi Guo, Yan Gao, Shi Han, Jian-Guang Lou, and Dongmei Zhang. 2022. HiTab: A hierarchical table dataset for question answering and natural language generation. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1094–1110, Dublin, Ireland. Association for Computational Linguistics.

DeepSeek-AI. 2025. DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning. *Nature*, 645(8081):633–638.

Xi Fang, Weijie Xu, Fiona Anting Tan, Ziqing Hu, Jiani Zhang, Yanjun Qi, Srinivasan H. Sengamedu, and Christos Faloutsos. 2024. Large language models (LLMs) on tabular data: Prediction, generation, and understanding - a survey. *Transactions on Machine Learning Research*.

GitHub. 2019. GitHub Flavored Markdown spec. https://github.github.com/gfm/. Version 0.29-gfm.

Google DeepMind. 2025. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities. *Preprint*, arXiv:2507.06261.

Tingxu Han, Zhenting Wang, Chunrong Fang, Shiyu Zhao, Shiqing Ma, and Zhenyu Chen. 2025. Token-budget-aware llm reasoning. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 24842–24855, Vienna, Austria. Association for Computational Linguistics.

Jonathan Herzig, Pawel Krzysztof Nowak, Thomas Müller, Francesco Piccinno, and Julian Eisenschlos. 2020. TaPas: Weakly supervised table parsing via pre-training. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 4320–4333, Online. Association for Computational Linguistics.

Hoyoun Jung and Kyung-Joong Kim. 2024. Discrete prompt compression with reinforcement learning. *IEEE Access*, 12:72578–72587.

LaTeX2e Unofficial Reference Manual Project. 2026. LaTeX2e: An unofficial reference manual — \multicolumn. https://latexref.xyz/_005cmulticolumn.html. Section 8.23.1; version of June 2026.

Liyao Li, Jiaming Tian, Hao Chen, Wentao Ye, Chao Ye, Haobo Wang, Ningtao Wang, Xing Fu, Gang Chen, and Junbo Zhao. 2025. LongTableBench: Benchmarking long-context table reasoning across real-world formats and domains. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 11927–11965, Suzhou, China. Association for Computational Linguistics.

Weizheng Lu, Jing Zhang, Ju Fan, Zihao Fu, Yueguo Chen, and Xiaoyong Du. 2025. Large language model for table processing: A survey. *Frontiers of Computer Science*, 19(2):192350.

Quinn McNemar. 1947. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157.

MDN Web Docs. 2026. How to write in Markdown. https://developer.mozilla.org/en-US/docs/MDN/Writing_guidelines/Howto/Markdown_in_MDN. Last modified 2026-01-04.

Meta AI. 2025. The llama 4 herd: The beginning of a new era of natively multimodal ai innovation. https://ai.meta.com/blog/llama-4-multimodal-intelligence/.

Microsoft. 2025. Phi-4-mini technical report: Compact yet powerful multimodal language models via mixture-of-LoRAs. *Preprint*, arXiv:2503.01743.

Julian Oestreich and Lydia Müller. 2025. Evaluating structured decoding for text-to-table generation: Evidence from three datasets. In *Proceedings of the First Workshop on Comparative Performance Evaluation: From Rules to Language Models*, pages 113–122, Varna, Bulgaria. INCOMA Ltd., Shoumen, Bulgaria.

OpenAI. 2025. Gpt-5 system card. https://cdn.openai.com/gpt-5-system-card.pdf.

Vaishali Pal, Andrew Yates, Evangelos Kanoulas, and Maarten de Rijke. 2023. MultiTabQA: Generating tabular answers for multi-table question answering. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 6322–6334, Toronto, Canada. Association for Computational Linguistics.

Panupong Pasupat and Percy Liang. 2015. Compositional semantic parsing on semi-structured tables. In *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics and the 7th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 1470–1480, Beijing, China. Association for Computational Linguistics.

Fletcher T. Penney. 2023. MultiMarkdown user's guide. https://fletcher.github.io/MultiMarkdown-6/MMD_Users_Guide.html. Version 6.6.0; revised 2023-05-25.

Qwen Team. 2025a. Qwen3 technical report. *Preprint*, arXiv:2505.09388.

Qwen Team. 2025b. Qwen3-vl technical report. *Preprint*, arXiv:2511.21631.

Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. 2016. SQuAD: 100,000+ questions for machine comprehension of text. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pages 2383–2392, Austin, Texas. Association for Computational Linguistics.

Pritika Ramu, Aparna Garimella, and Sambaran Bandyopadhyay. 2024. Is this a bad table? a closer look at the evaluation of table generation from text. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 22206–22216, Miami, Florida, USA. Association for Computational Linguistics.

Yuan Sui, Mengyu Zhou, Mingjie Zhou, Shi Han, and Dongmei Zhang. 2024. Table meets LLM: Can large language models understand structured table data? a benchmark and empirical study. In *Proceedings of the 17th ACM International Conference on Web Search and Data Mining*, WSDM '24, pages 645–654, New York, NY, USA. Association for Computing Machinery.

Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurélien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023. Llama: Open and efficient foundation language models. *Preprint*, arXiv:2302.13971.

Pieter van Oostrum, Øystein Bache, and Jerry Leichter. 2024. The `multirow`, `bigstrut` and `bigdelim` packages. `https://mirrors.ctan.org/macros/latex/contrib/multirow/multirow.pdf`. Version 2.9, released 2024-11-12.

WHATWG. 2026. HTML living standard: Tables. `https://html.spec.whatwg.org/multipage/tables.html`.

Pengzuo Wu, Yuhang Yang, Guangcheng Zhu, Chao Ye, Hong Gu, Xu Lu, Ruixuan Xiao, Bowen Bao, Yijing He, Liangyu Zha, Wentao Ye, Junbo Zhao, and Haobo Wang. 2025. RealHiTBench: A comprehensive realistic hierarchical table benchmark for evaluating LLM-based table analysis. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 7105–7137, Vienna, Austria. Association for Computational Linguistics.

Pengcheng Yin, Graham Neubig, Wen-tau Yih, and Sebastian Riedel. 2020. TaBERT: Pretraining for joint understanding of textual and tabular data. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 8413–8426, Online. Association for Computational Linguistics.

Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, Zilin Zhang, and Dragomir Radev. 2018. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 3911–3921, Brussels, Belgium. Association for Computational Linguistics.

Jintian Zhang, Yuqi Zhu, Mengshu Sun, Yujie Luo, Shuofei Qiao, Lun Du, Da Zheng, Huajun Chen, and Ningyu Zhang. 2025. Lightthinker: Thinking step-by-step compression. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 13307–13328, Suzhou, China. Association for Computational Linguistics.

Xuechen Zhang, Zijian Huang, Ege Onur Taga, Carlee Joe-Wong, Samet Oymak, and Jiasi Chen. 2024. Efficient contextual LLM cascades through budget-constrained policy learning. In *Proceedings of the 38th International Conference on Neural Information Processing Systems*, NIPS '24, Red Hook, NY, USA. Curran Associates Inc.

Victor Zhong, Caiming Xiong, and Richard Socher. 2017. Seq2sql: Generating structured queries from natural language using reinforcement learning. *Preprint*, arXiv:1709.00103.

Fengbin Zhu, Wenqiang Lei, Youcheng Huang, Chao Wang, Shuo Zhang, Jiancheng Lv, Fuli Feng, and Tat-Seng Chua. 2021. TAT-QA: A question answering benchmark on a hybrid of tabular and textual content in finance. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 3277–3287, Online. Association for Computational Linguistics.

# A Complexity Level Definitions

| Level | Required Features | Description | Example |
|---|---|---|---|
| **L1** | F1 | Atomic Grid | Simple data listing |
| **L2** | F1 + F2 | Headered Table | Standard table with headers |
| **L3** | F1 + F2 + F3 | Spanning Table | Table requiring cell merging |
| **L4** | F1 + F2 + F3 + F4 | Multi-level Header | Hierarchical header structure |
| **L5** | F1 + F2 + F3 + F4 + F5 | Block-partitioned | Same-schema section division |
| **L6** | F1 + F2 + F3 + F4 + F5 + F6 | Composite Table | Multi-schema combination |

Table 5: Complexity Level Definitions.

# B Additional Statistics of STB

## B.1 Complexity and Domain Distributions of STB

Tables 6 and 7 summarize the complexity-level and domain distributions of STB, respectively.

| **Level** | **#Tables** | **Share (%)** | **Avg. rows** | **Avg. cols** |
|---|---|---|---|---|
| L1 (Grid Only) | 96 | 8.82 | 4.33 | 3.15 |
| L2 (Basic Header) | 334 | 30.67 | 5.63 | 3.34 |
| L3 (Spanned) | 342 | 31.40 | 7.15 | 5.22 |
| L4 (Multi-level Header) | 164 | 15.06 | 8.84 | 6.65 |
| L5 (Section Stacking) | 87 | 7.99 | 12.66 | 4.20 |
| L6 (Composite) | 66 | 6.06 | 15.55 | 11.50 |
| **Total** | 1,089 | 100.00 | – | – |

Table 6: Complexity-level distribution and average table size of STB (tables containing all four intents: LOOKUP/COUNTING/COMPARISON/AGGREGATION). Average rows/cols aggregate all sub-tables for multi-table cases.

| **Domain** | **#Tables** | **Share (%)** |
|---|---|---|
| Scientific | 366 | 33.61 |
| Financial | 286 | 26.26 |
| Survey | 179 | 16.44 |
| Product | 171 | 15.70 |
| Schedule | 87 | 7.99 |
| **Total** | 1,089 | 100.00 |

Table 7: Domain distribution of STB (tables containing all four intents).

# C Formal Grammar (EBNF)

This appendix defines the **syntax** of LTEM using Extended Backus–Naur Form (EBNF). The grammar is a context-free grammar (CFG) and follows the notation of ISO/IEC 14977:1996.

### C.1 Notation Conventions

| Symbol | Meaning |
|---|---|
| = | Definition |
| , | Concatenation |
| \| | Alternation |
| [ ] | Optional (0 or 1 occurrence) |
| { } | Repetition (0 or more occurrences) |
| ( ) | Grouping |
| " " | Terminal string |
| '...' | Terminal string (alternative) |
| ? ? | Special sequence (e.g., regular expression) |
| (* *) | Comment |

Table 8: Notation conventions used in the EBNF specification.

### C.2 Document Structure

```
(* LTEM Document - Top Level *)
document = { table } ;

(* Table Structure *)
table = [ table_id_row ], header_section, separator_row, [ data_section ] ;

table_id_row = "|", "#", whitespace, table_id, whitespace, "|", newline ;

table_id = letter_or_underscore, { letter_or_digit_or_underscore } ;

header_section = row, { row } ;

data_section = row, { row } ;

separator_row = "|", separator_cell, { "|", separator_cell }, "|", newline ;

separator_cell = { "-" }, [ ":" ], { "-" } ;

row = "|", cell, { "|", cell }, "|", [ newline ] ;
```

### C.3 Cell Structure

```
(* Cell Definition *)
cell = cell_content, [ span_attribute ]
     | colspan_marker
     | rowspan_marker
     | empty_cell ;

empty_cell = { whitespace } ;

cell_content = { content_element } ;

content_element = plain_text
                | table_reference
                | inline_math
                | display_math
                | inline_list
                | line_break
                | checkbox ;

plain_text = ? any character except |, {, }, \, $, @, newline ? ;
```

## C.4 Span Syntax

```
(* Span Markers *)
colspan_marker = { whitespace }, "<<", { whitespace } ;

rowspan_marker = { whitespace }, "^^", { whitespace } ;

(* Span Attributes *)
span_attribute = "{", span_spec, { ",", span_spec }, "}" ;

span_spec = colspan_spec | rowspan_spec ;

colspan_spec = "c", ":", positive_integer ;

rowspan_spec = "r", ":", positive_integer ;

positive_integer = nonzero_digit, { digit } ;
```

## C.5 Reference Syntax (Phase 4)

```
(* Table References *)
table_reference = simple_reference
                | filtered_reference
                | column_reference
                | cell_reference ;

simple_reference = "@", table_id ;

filtered_reference = "@", table_id, "[", filter_condition, "]" ;

column_reference = "@", table_id, ".", column_name ;

cell_reference = "@", table_id, "[", row_index, "]", ".", column_name ;

filter_condition = column_name, comparison_op, value ;

comparison_op = "<" | ">" | "<=" | ">=" | "=" | "!=" ;

column_name = letter_or_underscore, { letter_or_digit_or_underscore } ;

row_index = digit, { digit } ;

value = number | string_literal ;

number = [ "-" ], digit, { digit }, [ ".", digit, { digit } ] ;

string_literal = "'", { ? any character except ' ? }, "'" ;
```

## C.6 Inline Content Syntax

```
(* Line Break *)
line_break = "\\", "\\" ;

(* Inline Math *)
inline_math = "$", math_content, "$" ;

display_math = "$$", math_content, "$$" ;

math_content = { ? any character except $ ? } ;

(* Inline Lists *)
inline_list = list_item, { line_break, list_item } ;

list_item = bullet_item | numbered_item | checkbox_item ;

bullet_item = "-", whitespace, item_text ;

numbered_item = digit, { digit }, ".", whitespace, item_text ;
```

```
checkbox_item = checkbox, whitespace, item_text ;

checkbox = "[", ( "x" | " " ), "]" ;

item_text = { ? any character except \\, |, newline ? } ;
```

### C.7 Terminal Definitions

```
(* Character Classes *)
letter = "A" | "B" | ... | "Z" | "a" | "b" | ... | "z" ;

digit = "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9" ;

nonzero_digit = "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9" ;

letter_or_underscore = letter | "_" ;

letter_or_digit_or_underscore = letter | digit | "_" ;

whitespace = " " | "\t" ;

newline = "\n" | "\r\n" ;
```

### C.8 Grammar Properties

| Property | Value | Implication |
|---|---|---|
| Chomsky Type | Type 2 (CFG) | Efficient parsing ($O(n^3)$ worst, O(n) practical) |
| Ambiguity | Unambiguous | Deterministic parsing |
| Left Recursion | None | LL parser compatible |
| Lookahead | LL(1) for most constructs | Simple recursive descent |
| Unicode | Supported in content | UTF-8 encoding |

Table 9: Summary of grammar properties.

### C.9 Lark Implementation

The following is a Lark parser grammar derived from the EBNF above:

```
// LTEM Grammar for Lark Parser
// Version 1.0

start: document

document: table+

table: table_id_row? header_section separator_row data_section?

table_id_row: "|" "#" TABLE_ID "|" NEWLINE

header_section: row+

data_section: row+

separator_row: "|" SEP_CELL ("|" SEP_CELL)* "|" NEWLINE
SEP_CELL: /:?-+:?/

row: "|" cell ("|" cell)* "|" NEWLINE?

cell: cell_content span_attr?
    | COLSPAN_MARKER
    | ROWSPAN_MARKER
    | EMPTY_CELL
```

```
cell_content: content_element+

content_element: TEXT
               | table_ref
               | INLINE_MATH
               | DISPLAY_MATH
               | LINE_BREAK
               | CHECKBOX

span_attr: "{" span_spec ("," span_spec)* "}"
span_spec: COLSPAN_ATTR | ROWSPAN_ATTR

COLSPAN_ATTR: "c:" POSITIVE_INT
ROWSPAN_ATTR: "r:" POSITIVE_INT
POSITIVE_INT: /[1-9][0-9]*/

COLSPAN_MARKER: /\s*<<\s*/
ROWSPAN_MARKER: /\s*\^\^\s*/
EMPTY_CELL: /\s*/

table_ref: REF_CELL | REF_COLUMN | REF_FILTERED | REF_SIMPLE
REF_SIMPLE: /@[a-zA-Z_][a-zA-Z0-9_]*/
REF_FILTERED: /@[a-zA-Z_][a-zA-Z0-9_]*\[[^\]]+\]/
REF_COLUMN: /@[a-zA-Z_][a-zA-Z0-9_]*\.[a-zA-Z_][a-zA-Z0-9_]*/
REF_CELL: /@[a-zA-Z_][a-zA-Z0-9_]*\[[0-9]+\]\.[a-zA-Z_][a-zA-Z0-9_]*/

INLINE_MATH: /\$[^$]+\$/
DISPLAY_MATH: /\$\$[^$]+\$\$/

LINE_BREAK: /\s*\\\\\s*/

CHECKBOX: /\[[ x]\]/

TABLE_ID: /[a-zA-Z_][a-zA-Z0-9_]*/
TEXT: /[^|{}\n\\$@]+/
NEWLINE: /\r?\n/

%import common.WS_INLINE
%ignore WS_INLINE
```

## D Prompt Templates

This appendix reports the prompt templates used for STB QA generation, QA verification, and table-understanding evaluation. Curly-brace fields denote runtime variables filled by the pipeline.

**STB QA Generation Prompt**

```
Create 1 table QA as JSON.
INTENT={intent}
Return ONLY: question, answer, targets. {targets_rule}
Rules: answerable ONLY from Cells; question must NOT mention [header,rowspan,colspan,merge,
    layout,structure,level,index,row,column]; answer must be short (number/short phrase), no
    explanation; Targets should list only data cells that directly contribute to the final
    answer. Compute the answer considering the units and symbols shown in the table.
Format: table_key(optional)|row,col,rowspan,colspan,H|D|text (H=header, D=data). {target_hint}
{intent_guidance}
{level_hint}
{multi_hint}
Cells:
{table_lines}
Output exactly: {output_schema}
```

The intent guidance field is instantiated as follows: LOOKUP reads one value for a specific item and attribute; AGGREGATION computes a numeric summary over multiple numeric cells; COUNTING

returns an integer count of items meeting a condition; and COMPARISON compares multiple numeric values and selects the higher, lower, best, or worst item.

**QA Verification Prompt**

```
You are a strict validator.
Answer the question using ONLY the table cells.
Return JSON ONLY: {"answer":"..."}.
No explanation. No extra keys. Answer must be short.

{fmt_line}

Question: {question}
Cells:
{table_lines}
```

**Table Understanding Evaluation Prompt**

```
You are an expert data analyst answering questions from tables. Use only the table; if answer
    missing, output N/A. Respond with ONLY the final answer value (no explanation, no prefix).
     If a number has commas, omit commas. Include units only if shown in the table.
Table format: {fmt}.{fmt_hint}

TABLE:
{table_text}

QUESTION: {question}
ANSWER :
```

For LTEM inputs, the evaluation prompt additionally includes the following guide:

**LTEM Guide Added to Evaluation Prompt**

```
LTEM guide: '<<' = copy nearest non-<< cell to the left; '^^' = copy nearest non-^^ cell above
    ; {c:N}/{r:N} = explicit col/row spans; '|---|' separates headers; '\\' is a newline.
    Resolve merges/spans first and answer from the resolved table.
```

All token counts reported in the paper are computed from the stored prompts using the `o200k_base` tokenizer.

## E STB Generation Pipeline

STB generation follows a controlled three-stage pipeline. In the first stage, a rule-based structural generator samples the complexity level, domain, row/column count, and the structural operations (F1–F6) required for each table. In the second stage, large language models populate each table with domain-appropriate headers and cell values while preserving the sampled structure. In the third stage, question–answer pairs are generated for four intent types (LOOKUP, COUNTING, COMPARISON, AGGREGATION) and filtered by agreement between two independent model families. Prompt templates used in each stage are provided in Appendix D.

## F HiTab Overall Results

Per-format performance on the HiTab test split is shown below. The STB overall results appear as Table 2 in the main body.

| Format | nEM↑ | TokenF1↑ | NumEM↑ | PromptTok↓ | $TNP_{adj}$ ↑ | Rank |
|---|---|---|---|---|---|---|
| **HTML** | **0.295**$_{\pm 0.001}$ | 0.331$_{\pm 0.002}$ | 0.519$_{\pm 0.001}$ | 1310 | 0.225 | **1** |
| LaTeX | 0.292$_{\pm 0.003}$ | 0.333$_{\pm 0.003}$ | 0.527$_{\pm 0.002}$ | 966 | 0.303 | 2 |
| LTEM | 0.289$_{\pm 0.005}$ | 0.329$_{\pm 0.004}$ | 0.504$_{\pm 0.004}$ | 1101 | 0.262 | 3 |
| MD | 0.285$_{\pm 0.003}$ | 0.322$_{\pm 0.004}$ | 0.515$_{\pm 0.003}$ | 1078 | 0.264 | 4 |
| MMD | 0.281$_{\pm 0.001}$ | 0.320$_{\pm 0.001}$ | 0.520$_{\pm 0.002}$ | 1058 | 0.266 | 5 |
| CSV | 0.266$_{\pm 0.004}$ | 0.306$_{\pm 0.003}$ | 0.515$_{\pm 0.003}$ | 770 | 0.345 | 6 |

Table 10: Overall performance on the HiTab test split (515 hierarchical tables). HTML achieves the highest nEM, followed by LaTeX. The ranking of LaTeX reverses compared to STB (rank 6→2), suggesting format effectiveness is dataset-dependent.

**Table Generation success rates.** Table 11 reports the Excellent-rate breakdown corresponding to the Table Generation results in Table 4.

| Format | Excellent (%) |
|---|---|
| **HTML** | **76.2** |
| LaTeX | 29.3 |
| MultiMarkdown | 35.4 |
| LTEM | 25.8 |
| CSV | 25.4 |
| Markdown | 22.5 |

Table 11: Excellent-rate breakdown for Table Generation on HiTab. Excellent denotes ANLS $\geq 0.9$; the values use the same evaluation setting as Table 4.

## G Supplementary Analyses

This appendix consolidates additional fine-grained analyses that characterize format behavior beyond the aggregate results in the main paper. Unless otherwise stated, the reported STB understanding numbers are normalized exact match (nEM) over the same evaluation protocol as the main paper. These results are intended to clarify where format differences arise rather than to replace the primary aggregate comparisons in the main Results section.

### G.1 Fine-grained STB Understanding Analyses

**Level- and feature-wise Markdown–LTEM comparison.** Table 12 compares Markdown and LTEM across the STB complexity levels. The nEM differences are small, with LTEM slightly higher at each level. Invalid rates are low for both formats and do not show a uniform level-wise advantage. This supports a conservative interpretation: LTEM's main distinction is explicit structural representation rather than a uniform large QA gain.

| Level | N | MD nEM | LTEM nEM | Δ | Invalid MD/LTEM |
|---|---|---|---|---|---|
| L1 | 96 | 0.863 | 0.866 | +0.003 | 0.05/0.05 |
| L2 | 334 | 0.845 | 0.847 | +0.002 | 0.22/0.13 |
| L3 | 342 | 0.787 | 0.791 | +0.004 | 0.11/0.16 |
| L4 | 164 | 0.735 | 0.737 | +0.002 | 0.17/0.19 |
| L5 | 87 | 0.858 | 0.863 | +0.004 | 0.00/0.00 |
| L6 | 66 | 0.788 | 0.793 | +0.005 | 0.28/0.20 |

Table 12: Level-wise comparison of Markdown and LTEM on STB. Invalid rates are percentages.

The same pattern appears when conditioning on structural features (Table 13). The main distinction starts at F3, cell merging: Markdown can only approximate such tables through visual alignment or repeated text, whereas LTEM explicitly marks span extent.

| Feature slice | N | MD nEM | LTEM nEM | Δ | Invalid MD/LTEM |
|---|---|---|---|---|---|
| F2 | 1,057 | 0.809 | 0.812 | +0.003 | 0.16/0.14 |
| F3 | 593 | 0.783 | 0.787 | +0.004 | 0.11/0.15 |
| F4 | 345 | 0.762 | 0.766 | +0.004 | 0.17/0.20 |
| F5 | 87 | 0.858 | 0.863 | +0.004 | 0.00/0.00 |
| F6 | 66 | 0.788 | 0.793 | +0.005 | 0.28/0.20 |

Table 13: Feature-conditioned Markdown–LTEM comparison on STB. F2–F6 denote operational slices in which the corresponding structural feature is observed; the slices may overlap and are not cumulative level subsets. Invalid rates are percentages.

### G.2 LTEM Design-Choice Ablation

To examine the sensitivity of Table QA performance to LTEM's surface syntax, we compare the proposed forms of five elements with structure-preserving alternatives on matched STB instances. For each comparison, table content, questions, reference answers, and the evaluation protocol are held fixed, and only the notation of the target element is varied. Experiments use GPT-5, and paired differences are assessed using exact McNemar tests.

| Element ($N$) | LTEM | Alternative | nEM (%) | $p$ |
|---|---|---|---|---|
| Colspan (400) | « | `{c:N}` | 77.8 / 78.8 | .481 |
| Rowspan (348) | ^^ | `{r:N}` | 75.6 / 76.7 | .454 |
| Header scope (288) | Separator | Header count | 75.3 / 77.1 | .302 |
| Section label (348) | Marker row | Attribute row | 85.6 / 85.9 | 1.000 |
| Table ID (264) | In-grid row | MD heading | 83.3 / 82.2 | .453 |

Table 14: Element-wise LTEM design-choice ablation on STB using GPT-5. Each row compares the LTEM form with a structure-preserving alternative on the same QA pairs ($N$). nEM is reported as LTEM/alternative; $p$-values are from exact McNemar tests.

None of the five paired comparisons reaches statistical significance (all $p \geq .302$); the ablation therefore does not identify a preferred surface form on the basis of QA accuracy. LTEM adopts the proposed forms in accordance with the Markdown Compatibility and Linear Readability principles.

### G.3 Structure-conditioned Accuracy

We stratify L1–L5 single-table examples by span-continuation and repeated-value ratios, using None/Low/Medium/High cutoffs of 0/.15/.35 and 0/.30/.60, respectively. Table 15 reports descriptive nEM results. Across the larger slices, LTEM remains comparable to HTML and Markdown, while LaTeX is generally lower; the small High buckets are reported without broad generalization.

| Condition | Bucket | HTML | LaTeX | LTEM | MD |
|---|---|---|---|---|---|
| Merged density | None (430) | 0.852 | 0.811 | 0.852 | 0.849 |
| Merged density | Low (318) | 0.781 | 0.750 | 0.781 | 0.783 |
| Merged density | Medium (265) | 0.792 | 0.759 | 0.796 | 0.786 |
| Merged density | High (10) | 0.735 | 0.725 | 0.725 | 0.725 |
| Duplicate-value | None (668) | 0.823 | 0.789 | 0.822 | 0.820 |
| Duplicate-value | Low (246) | 0.773 | 0.741 | 0.778 | 0.775 |
| Duplicate-value | Medium (98) | 0.841 | 0.792 | 0.846 | 0.837 |
| Duplicate-value | High (11) | 0.829 | 0.782 | 0.836 | 0.833 |

Table 15: Structure-conditioned nEM on STB L1–L5 by merged-cell density and duplicate-value ambiguity. Table counts are shown in parentheses.

### G.4 Failure Modes and Question Types

Refusal and Invalid rates are low for most formats. LaTeX nevertheless shows substantially higher Invalid and logged-error rates than the other text formats (Table 16).

| Format | nEM | Refusal | Invalid | Logged error |
|---|---|---|---|---|
| CSV | 0.808 | 0.22 | 0.09 | 0.11 |
| HTML | 0.812 | 0.30 | 0.11 | 0.15 |
| LaTeX | 0.777 | 0.38 | 2.67 | 3.55 |
| LTEM | 0.813 | 0.15 | 0.14 | 0.18 |
| MD | 0.810 | 0.46 | 0.15 | 0.20 |
| MMD | 0.810 | 0.34 | 0.15 | 0.19 |

Table 16: Failure diagnostics on STB. Refusal and Invalid are calculated among nEM-incorrect outputs; Logged error is recorded independently and may overlap. Values other than nEM are percentages.

Table 17 reports descriptive nEM by question type. Aggregation is the most difficult type across formats; we do not infer a format-specific question-type advantage from this breakdown.

| Format | Aggregation | Comparison | Counting | Lookup |
|---|---|---|---|---|
| CSV | 0.529 | 0.949 | 0.919 | 0.834 |
| HTML | 0.531 | 0.954 | 0.925 | 0.838 |
| LaTeX | 0.513 | 0.915 | 0.891 | 0.788 |
| LTEM | 0.530 | 0.954 | 0.925 | 0.843 |
| MD | 0.529 | 0.955 | 0.923 | 0.831 |
| MMD | 0.532 | 0.955 | 0.924 | 0.830 |

Table 17: Question-type nEM breakdown on STB.

### G.5 Fine-tuning Pilot

To assess whether small instruction-tuned models can adapt to LTEM with limited supervision, we conducted a LoRA pilot using LTEM-formatted STB QA examples. Tables were partitioned across the training, validation, and test sets, yielding 3,472, 424, and 460 examples, respectively. We used LoRA rank 16, alpha 32, dropout 0.05, two epochs, a learning rate of $2 \times 10^{-4}$, and a maximum sequence length of 4096.

Table 18 shows substantial gains for all three backbones on held-out STB tables. These results indicate that LTEM-formatted table QA is amenable to adaptation with limited in-domain supervision. Because format exposure and task supervision are introduced jointly, however, the pilot does not isolate their individual contributions. The evaluation is limited to 4B-scale backbones, the STB distribution, and a single set of LoRA hyperparameters.

| Backbone | Base nEM | LoRA nEM | $\Delta$ nEM |
|---|---|---|---|
| Qwen3-4B-Instruct | 0.489 | 0.763 | +0.274 |
| Phi-4-mini | 0.539 | 0.737 | +0.198 |
| Gemma-3-4B-IT | 0.361 | 0.707 | +0.346 |

Table 18: LTEM LoRA pilot results on held-out STB tables using normalized Exact Match (nEM) as defined in the main evaluation.

For Qwen3-4B-Instruct, the largest gains occur on comparison and aggregation questions (Table 19), while lookup and counting improve more modestly.

| Slice | Base nEM | LoRA nEM | Δ |
|---|---|---|---|
| LOOKUP | 0.817 | 0.983 | +0.165 |
| COUNTING | 0.609 | 0.696 | +0.087 |
| AGGREGATION | 0.148 | 0.478 | +0.330 |
| COMPARISON | 0.383 | 0.896 | +0.513 |
| L1 | 0.500 | 0.773 | +0.273 |
| L2 | 0.522 | 0.750 | +0.228 |
| L3 | 0.457 | 0.800 | +0.343 |
| L4 | 0.529 | 0.765 | +0.235 |
| L5 | 0.375 | 0.725 | +0.350 |
| L6 | 0.531 | 0.688 | +0.156 |

Table 19: Qwen3-4B LTEM LoRA breakdown by question type and complexity level.

## H McNemar's Test for LTEM vs. LaTeX on STB

To assess paired differences between LTEM and LaTeX on STB, we applied McNemar's test to matched predictions within each of three runs and combined the resulting p-values using Fisher's method.

| Model | $n_{01}$ | $n_{10}$ | Fisher $p$ | Winner |
|---|---|---|---|---|
| DeepSeek-R1 | 196 | 343 | $5.4\times10^{-9}$*** | LTEM |
| Llama-4-Maverick | 343 | 441 | 0.0056** | LTEM |
| Gemini-2.5-Pro | 253 | 275 | 0.410 (ns) | — |
| GPT-5 | 246 | 353 | $2.1\times10^{-4}$*** | LTEM |
| Qwen3-235B | 398 | 2,377 | $<10^{-100}$*** | LTEM |

Table 20: McNemar test comparing LTEM vs. LaTeX on STB. $n_{01}$ denotes LTEM-wrong & LaTeX-correct, and $n_{10}$ denotes LTEM-correct & LaTeX-wrong. The discordant counts are summed across three runs, and the p-values are Fisher-combined. Significance: *** $p<0.001$, ** $p<0.01$, * $p<0.05$.

## I Translation Disclosure

We used AI tools as auxiliary aids for some translation. All translations were reviewed and revised by the authors, and the final manuscript reflects our verification of factual accuracy and phrasing.